Άσκηση 1 [30 μονάδες]

Έστω $T_1(n)$, $T_2(n)$ και $T_3(n)$ το πλήθος εκτελέσεων της εντολής `z = z + 1` από τις παρακάτω ρουτίνες. Υπολογίστε τα $T_1(n)$, $T_2(n)$ και $T_3(n)$ (δηλαδή βρείτε κλειστό τύπο για τις συναρτήσεις $T_1(), T_2(), T_3()$).

```
procedure PuzzleA(integer n)
  for (i = 1; i <= n; i++)
    for (j = 1; j <= i; j++)
      for (k = 1; k <= j; k++)
        z = z + 1;

procedure PuzzleB(integer n)
  for (i = 1; i <= n; i++)
    if (i % 2 == 1)
      for (j = 1; j <= i; j++)
        z = z + 1;

procedure PuzzleC(integer n)
  for (i = 1; i <= n; i++)
    for (j = 1; j <= n*i; j++)
      z = z + 1;
```

Λύση

(α) Το πλήθος των εκτελέσεων είναι

$$T_1(n) = \sum_{i=1}^{n}\sum_{j=1}^{i}\sum_{k=1}^{j} 1 = \sum_{i=1}^{n}\sum_{j=1}^{i} j = \sum_{i=1}^{n} \frac{i(i+1)}{2} = \frac{1}{2}\left(\sum_{i=1}^{n} i^2 + \sum_{i=1}^{n} i\right).$$

Χρησιμοποιώντας τους γνωστούς τύπους $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$ και $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$,

$$T_1(n) = \frac{1}{2}\left(\frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2}\right) = \frac{n(n+1)(n+2)}{6}.$$

Άρα $\boxed{T_1(n) = \dfrac{n(n+1)(n+2)}{6}}$ (τετραεδικοί αριθμοί) και $T_1(n) = \Theta(n^3)$.

(β) Η εντολή τρέχει μόνο για περιττά i, και τότε τρέχει i φορές:

$$T_2(n) = \sum_{\substack{1\le i\le n \\ i \text{ περιττό}}} i = 1 + 3 + 5 + \cdots + \left(2\lfloor \tfrac{n+1}{2} \rfloor - 1\right).$$

Το άθροισμα των πρώτων m περιττών είναι m^2, όπου $m = \lfloor \frac{n+1}{2} \rfloor$. Επομένως

$$\boxed{T_2(n) = \left\lfloor \frac{n+1}{2} \right\rfloor^2} = \begin{cases} \left(\dfrac{n}{2}\right)^2, & n \text{ άρτιος}, \\ \left(\dfrac{n+1}{2}\right)^2, & n \text{ περιττός}. \end{cases}$$

Συνεπώς $T_2(n) = \Theta(n^2)$.

(γ) Για κάθε i η εσωτερική επανάληψη τρέχει $n \cdot i$ φορές, άρα

$$T_3(n) = \sum_{i=1}^{n} n\, i = n \sum_{i=1}^{n} i = n \cdot \frac{n(n+1)}{2} = \frac{n^2(n+1)}{2}.$$

Άρα $\boxed{T_3(n) = \dfrac{n^2(n+1)}{2}}$ και $T_3(n) = \Theta(n^3)$.

Άσκηση 2 [25 μονάδες]

(α) (10Μ) Μελετήστε την ασυμπτωτική πολυπλοκότητα (βάσει του συμβολισμού Θ) της συνάρτησης

$$F(n) = 5n^3 + 2n^2 \log n - 3n(\log n)^2 + 7.$$

(β) (15Μ) Διατάξτε τις ακόλουθες συναρτήσεις κατά φθίνουσα ασυμπτωτική ανάπτυξη, δηλαδή βρείτε μια ακολουθία $g_1, \ldots, g_{15}$ που ικανοποιεί

$$g_1 = \Omega(g_2), \quad g_2 = \Omega(g_3), \quad \ldots, \quad g_{14} = \Omega(g_{15}),$$

και στη συνέχεια διαμερίστε σε κλάσεις ισοδυναμίας ως προς το $\Theta(\cdot)$ (ίδια τάξη αν και μόνο αν $f(n) = \Theta(g(n))$). Όλοι οι λογάριθμοι θεωρούνται με βάση 2.

Συναρτήσεις:

$n \log n$, $log2n$, $n^{1/2}$, $2^{\log n}$, n^3, $n^2 \log^5 n$, $\log(n!)$, n^{logn}, 3^n, $n^{1.5}$, $\log n$, 2^n, n, n^2, $(n+1)!$.

Λύση

(α) Δίνεται:

$$F(n) = 5n^3 + 2n^2 \log n - 3n(\log n)^2 + 7.$$

Θέλουμε να βρούμε ασυμπτωτικό φράγμα $\Theta(\cdot)$ για τη $F(n)$.

Ανώτερο φράγμα. Επειδή $\log n \le n$ για κάθε $n \ge 1$, έχουμε:

$$(\log n)^2 \le n^2 \log n \le n^3.$$

Άρα:

$$F(n) = 5n^3 + 2n^2 \log n - 3n(\log n)^2 + 7 \le 5n^3 + 2n^2 \log n + 7 \le 5n^3 + 2n^3 + 7 \le 7n^3 + 7.$$

Για $n \ge 2$ ισχύει $7 \le n^3$, συνεπώς

$$F(n) \le 8n^3, \quad n \ge 2. \tag{1}$$

Κατώτερο φράγμα. Επειδή $(\log n)^2 \le n^2$, προκύπτει

$$-3n(\log n)^2 \ge -3n^3.$$

Άρα:

$$F(n) \ge 5n^3 - 3n^3 = 2n^3, \quad n \ge 1. \tag{2}$$

Συμπέρασμα. Από τις σχέσεις (1) και (2) έχουμε

$$2n^3 \le F(n) \le 8n^3, \quad n \ge 2,$$

οπότε

$$\boxed{F(n) = \Theta(n^3).}$$

(β) Διάταξη κατά φθίνουσα ανάπτυξη και κλάσεις Θ

Υποθέσεις & χρήσιμα: Θεωρούμε όλους τους λογαρίθμους με βάση 2, δηλ. $\log \equiv \log_2$. Τότε $2^{\log n} = n$, ενώ $n^{\log n} = 2^{(\log n)^2}$ και $\log(2n) = \Theta(\log n)$. Επίσης $\log(n!) = \Theta(n \log n)$.

Φθίνουσα ασυμπτωτική διάταξη $g_1 \succeq g_2 \succeq \cdots \succeq g_{15}$**:**

$$
\begin{aligned}
g_1 &= (n+1)!,\\
g_2 &= 3^n,\\
g_3 &= 2^n,\\
g_4 &= n^{\log n},\\
g_5 &= n^3,\\
g_6 &= n^2(\log n)^5,\\
g_7 &= n^2,\\
g_8 &= n^{1.5},\\
g_9 &= n \log n,\\
g_{10} &= \log(n!),\\
g_{11} &= 2^{\log n} = n,\\
g_{12} &= n,\\
g_{13} &= n^{1/2},\\
g_{14} &= \log(2n),\\
g_{15} &= \log n.
\end{aligned}
$$

Κλάσεις ισοδυναμίας ως προς $\Theta(\cdot)$**:**

$$
\begin{aligned}
&\{(n+1)!\},\ \{3^n\},\ \{2^n\},\ \{n^{\log n}\},\ \{n^3\},\ \{n^2(\log n)^5\},\ \{n^2\},\ \{n^{1.5}\},\\
&\{n \log n,\ \log(n!)\},\ \{n,\ 2^{\log n}\},\ \{n^{1/2}\},\ \{\log(2n),\ \log n\}.
\end{aligned}
$$

Σχόλιο ορθότητας (σύντομα):

$$(n+1)! \gg a^n\ (\forall a > 1) \gg n^{\log n} = 2^{(\log n)^2} \gg n^k \gg n^\alpha \log^c n \gg n^\beta \gg \log n,$$

με $k > \alpha > 1 > \beta > 0$ και $c > 0$.

Άσκηση 3 [20 μονάδες]

Εκφώνηση

(α) (10Μ) Επιλύστε με επαναληπτική αντικατάσταση την αναδρομική σχέση

$$T(n) = 3\,T\Big(\frac{n}{2}\Big) + n^2, \qquad T(1) = 1,$$

και δώστε αυστηρό ασυμπτωτικό φράγμα.

(β) (10Μ) Σχεδιάστε (αναλύστε ανά επίπεδο) το δένδρο αναδρομής για την

$$T(n) = T\Big(\frac{n}{4}\Big) + T\Big(\frac{n}{2}\Big) + n$$

και μελετήστε την ασυμπτωτική της συμπεριφορά.

Λύση

(α) $T(n) = 3T(n/2)+n^2$, $T(1) = 1$ (επαναληπτική αντικατάσταση) Αναπτύσσουμε για k επίπεδα:

$$\begin{aligned} T(n) &= 3\,T\Big(\frac{n}{2}\Big) + n^2 \\ &= 3\Big(3\,T\Big(\frac{n}{2^2}\Big) + \big(\tfrac{n}{2}\big)^2\Big) + n^2 = 3^2 T\Big(\frac{n}{2^2}\Big) + n^2\Big(1+\frac{3}{4}\Big) \\ &= 3^k T\Big(\frac{n}{2^k}\Big) + n^2 \sum_{i=0}^{k-1} \Big(\frac{3}{4}\Big)^i . \end{aligned}$$

Παίρνουμε $k = \log_2 n$ ώστε $\frac{n}{2^k} = 1$ και $T(1) = 1$:

$$T(n) = 3^{\log_2 n} \cdot 1 + n^2 \sum_{i=0}^{\log_2 n - 1} \Big(\frac{3}{4}\Big)^i .$$

Το άθροισμα είναι γεωμετρική πρόοδος:

$$\sum_{i=0}^{\log_2 n-1} \Big(\frac{3}{4}\Big)^i = \frac{1-\big(\frac{3}{4}\big)^{\log_2 n}}{1-\frac{3}{4}} = 4\Big(1 - n^{\log_2(3/4)}\Big).$$

Άρα

$$T(n) = n^{\log_2 3} + 4n^2\Big(1 - n^{\log_2(3/4)}\Big) = 4n^2 - 4n^{\,2+\log_2(3/4)} + n^{\log_2 3}.$$

Επειδή $\log_2(3/4) = \log_2 3 - 2 < 0$, ο όρος $n^{\,2+\log_2(3/4)} = n^{\log_2 3}$ και ο τελευταίος είναι χαμηλότερης τάξης από n^2. Συνεπώς

$$\boxed{T(n) = \Theta(n^2)}.$$

Μάλιστα, από τον ρητό τύπο παίρνουμε για κάθε $n \geq 1$ ότι $n^2 \leq T(n) \leq 5n^2$, άρα το φράγμα είναι αυστηρό.

(β) $T(n) = T(n/4) + T(n/2) + n$ (δένδρο αναδρομής & ασυμπτωτική) Αναλύουμε το συνολικό κόστος ανά επίπεδο του δένδρου.

Επίπεδο 0 (ρίζα): κόστος n.
Επίπεδο 1: δύο κόμβοι με μεγέθη $n/2$ και $n/4$, άρα κόστος $\frac{n}{2} + \frac{n}{4} = \frac{3}{4}n$.
Επίπεδο 2: κάθε κόμβος διασπάται με τους ίδιους λόγους, και το συνολικό κόστος πολλαπλασιάζεται ξανά με $\frac{3}{4}$, άρα $\big(\frac{3}{4}\big)^2 n$.
Γενικά, στο επίπεδο i το κόστος είναι

$$\text{Κόστος επιπέδου } i = \Big(\frac{3}{4}\Big)^i n.$$

Το μέγιστο βάθος είναι $O(\log n)$ (ακολουθώντας πάντα το «μισό» υποπρόβλημα). Άρα το άθροισμα των ενδιάμεσων επιπέδων είναι γεωμετρικό:

$$\sum_{i=0}^{\lfloor \log n \rfloor} \Big(\frac{3}{4}\Big)^i n \;\leq\; \frac{n}{1-\frac{3}{4}} \;=\; 4n.$$

Οι συνεισφορές των φύλλων είναι $O(1)$ ανά φύλλο και ο αριθμός τους είναι πολυωνυμικά μικρότερος από $n/(e)$ ανά επίπεδο· συνολικά δεν αλλάζουν την τάξη μεγέθους. Εναλλακτικά, επαλήθευση με επαγωγική *δοκιμή* $T(n) \leq cn$:

$$T(n) \leq c\frac{n}{4} + c\frac{n}{2} + n = \frac{3c}{4}n + n \leq cn \quad \text{για } c \geq 4.$$

Και προφανώς $T(n) \geq n$, άρα:

$$\boxed{T(n) = \Theta(n)}.$$

Άσκηση 4 [30 μονάδες]

Να σχεδιάσετε έναν αναδρομικό αλγόριθμο `FindMax(A[], i, j)` που επιστρέφει το μέγιστο στοιχείο ενός πίνακα $A[i..j]$.

(α) [10Μ] Ψευδοκώδικας

```
function FindMax(A, i, j):
  //                      : 0 <= i <= j < length(A)
  if i == j:
    return A[i]
  mid = floor((i + j) / 2)
  leftMax  = FindMax(A, i, mid)
  rightMax = FindMax(A, mid + 1, j)
  if leftMax >= rightMax:
    return leftMax
  else:
    return rightMax
```

Σχόλια: Ο αλγόριθμος 'διαίρει-και-βασίλευε' χωρίζει το διάστημα $[i..j]$ σε δύο σχεδόν ίσα υποδιαστήματα, βρίσκει αναδρομικά τα μέγιστα και επιστρέφει το μέγιστο των δύο.

(β) [5Μ] Σχέση αναδρομής χρόνου εκτέλεσης

Θέτοντας $n = j - i + 1$ το μέγεθος του υποπίνακα, ο χρόνος $T(n)$ ικανοποιεί:

$$T(1) = \Theta(1), \qquad T(n) = T\Big(\Big\lceil \frac{n}{2} \Big\rceil\Big) + T\Big(\Big\lfloor \frac{n}{2} \Big\rfloor\Big) + \Theta(1) \quad \text{για } n > 1.$$

Ισοδύναμα (για απλότητα): $T(n) = 2T(n/2) + \Theta(1)$ για n δύναμη του 2.

(γ) [5Μ] Επίλυση της αναδρομής και ασυμπτωτική συμπεριφορά

Χρησιμοποιώντας δένδρο αναδρομής :

$$T(n) = 2T(n/2) + \Theta(1) \quad \Longrightarrow \quad T(n) = \Theta(n).$$

Άρα:

$$\boxed{T(n) = O(n)} \quad \text{και} \quad \boxed{T(n) = \Omega(n)} \quad \Rightarrow \quad \boxed{T(n) = \Theta(n)}.$$

(δ) [5Μ] Ιχνήλατηση για $\texttt{FindMax}([4,7,1,9,2],0,4)$

Θα δείξουμε τις κλήσεις/επιστροφές (με σύγκριση στο ανέβασμα):

$\texttt{FindMax}(A,0,4) \quad (\text{mid}=2)$

$$\rightarrow \texttt{FindMax}(A,0,2)\ (\text{mid}=1) \ \rightarrow \begin{cases} \texttt{FindMax}(A,0,1)\ (\text{mid}=0) \rightarrow \begin{cases} \texttt{FindMax}(A,0,0)=4 \\ \texttt{FindMax}(A,1,1)=7 \end{cases} \\ \max(4,7)=7 \\ \texttt{FindMax}(A,2,2)=1 \end{cases}$$

$$\max(7,1)=7$$

$$\rightarrow \texttt{FindMax}(A,3,4)\ (\text{mid}=3) \rightarrow \begin{cases} \texttt{FindMax}(A,3,3)=9 \\ \texttt{FindMax}(A,4,4)=2 \end{cases}$$

$$\max(9,2)=9$$

Result: $\max(7,9)=\boxed{9}$.

Αριθμός συγκρίσεων: Σε κάθε εσωτερικό κόμβο γίνεται μία σύγκριση `leftMax >= rightMax`. Για $n=5$ υπάρχουν $n-1=4$ τέτοιοι κόμβοι, άρα 4 συγκρίσεις (η γνωστή κατώτερη/ανώτερη τιμή για εύρεση μέγιστου με συγκρίσεις είναι $n-1$).

(ε) [5Μ] Χωρική πολυπλοκότητα

Το βάθος της αναδρομής είναι το ύψος του δένδρου: $\lfloor \log_2 n \rfloor + 1$ στο καλύτερο/μέσο σενάριο (ισορροπημένη διάσπαση). Κάθε επίπεδο κρατά $O(1)$ πρόσθετη πληροφορία (δείκτες/τιμές).

$$\boxed{\text{Βοηθητικός χώρος στο στοίβο} = O(\log n)}.$$

Ο ίδιος ο πίνακας A δεν αντιγράφεται (πρόσβαση με δείκτες/όρια), οπότε ο *συνολικός επιπλέον* χώρος παραμένει $O(\log n)$.

Πανεπιστήμιο Κρήτης, Τμήμα Επιστήμης Υπολογιστών

HY240: Δομές Δεδομένων

Χειμερινό Εξάμηνο – Ακαδημαϊκό Έτος 2025-26

2η Σειρά Ασκήσεων

Ο ψευδοκώδικας είναι βασισμένος στην C, αλλά και οποιαδήποτε άλλη μορφή είναι δεκτή και εξίσου σωστή.

Άσκηση 1 [25 μονάδες]

α. Ένα γράμμα σημαίνει enqueue και ένας αστερίσκος σημαίνει dequeue στην ακόλουθη ακολουθία χαρακτήρων:

A B C * D E * * F G * * H * *

Δώστε την ακολουθία των τιμών που επιστρέφονται από τις πράξεις dequeue, όταν αυτή η ακολουθία πράξεων εκτελείται σε μια αρχικά κενή ουρά.

[5M]

Λύση

A, B, C, D, E, F, G

β. Δίνονται οι εξής πράξεις πάνω σε μια διπλά συνδεδεμένη λίστα L. Να γράψετε ψευδοκώδικες για:

1) Find(L, k) που επιστρέφει δείκτη στον κόμβο με τιμή k, αν υπάρχει, αλλιώς NULL.

[5M]

2) InsertAfter(L, k1, k2) που εισάγει νέο κόμβο με τιμή k2 αμέσως μετά τον κόμβο που περιέχει k1.

[5M]

3) DeleteValue(L, k) που διαγράφει τον πρώτο κόμβο που περιέχει k. Αν δεν υπάρχει, να μην αλλάζει η λίστα.

[10M]

Λύση

1)

```
DNode* Find(DList* L, int k) {
    DNode* p = L->head;
    while (p != NULL) {
        if (p->key == k) return p;
        p = p->next;
    }
    return NULL;
}
```

2)

```
void InsertAfter(DList* L, int k1, int k2) {
    DNode* p = Find(L, k1);
    if (p == NULL) return;

    DNode* n = (DNode*)malloc(sizeof(DNode));
    n->key = k2;

    n->prev = p;
    n->next = p->next;

    if (p->next != NULL) {
        p->next->prev = n;
    } else {
        L->tail = n;
    }
    p->next = n;
}
```

3)

```
void DeleteValue(DList* L, int k) {
    DNode* p = Find(L, k);
    if (p == NULL) return;

    if (p->prev != NULL) p->prev->next = p->next;
    else                 L->head = p->next;

    if (p->next != NULL) p->next->prev = p->prev;
    else                 L->tail = p->prev;

    free(p);
}
```

Άσκηση 2 [30 μονάδες]

α. Χρησιμοποιώντας μόνο τις συναρτήσεις της βιβλιοθήκης στοίβας και ουράς, σχεδιάστε μη-αναδρομική συνάρτησηTransferAlternate(Stack S, Queue Q) που μεταφέρει εναλλάξ τα στοιχεία της στοίβας S και της ουράς Q σε μια νέα στοίβα R, έτσι ώστε: a) το πρώτο στοιχείο της R να προέρχεται από την S, b) το δεύτερο από την Q, c) και ούτω καθεξής.
Αν κάποια από τις δύο δομές αδειάσει νωρίτερα, τα υπόλοιπα στοιχεία της άλλης αντιγράφονται στη συνέχεια της R.
Η συνάρτηση να έχει πολυπλοκότητα O(n + m).

[15M]

Λύση

```
Stack TransferAlternate(Stack* S, Queue* Q) {
    Stack R; R.top = NULL;
    bool takeFromS = true;

    // 1. Εναλλαγή όσο υπάρχουν στοιχεία ΚΑΙ στις δύο δομές
    while (!stack_empty(S) && !queue_empty(Q)) {
        if (takeFromS) {
            int x = pop(S);
            push(&R, x);
        } else {
            int y = dequeue(Q);
            push(&R, y);
        }
        takeFromS = !takeFromS; // Εναλλαγή
    }

    // 2. Αντιγραφή των υπολοίπων στοιχείων της S (αν υπάρχουν)
    while (!stack_empty(S)) {
        int x = pop(S);
        push(&R, x);
    }

    // 3. Αντιγραφή των υπολοίπων στοιχείων της Q (αν υπάρχουν)
    while (!queue_empty(Q)) {
        int y = dequeue(Q);
        push(&R, y);
    }

    // Η πολυπλοκότητα είναι O(n + m).
    return R;

}
```

β. Να αναπτύξετε αλγόριθμο που διαβάζει μια ακολουθία από παρενθέσεις (π.χ. "(([]){}())") και ελέγχει αν είναι σωστά ζευγαρωμένες. Επιτρέπεται η χρήση μόνο μίας στοίβας και βοηθητικών μεταβλητών.

[15Μ]

Λύση

```
static inline bool matches(char open, char close) {
    return (open=='(' && close==')') ||
           (open=='[' && close==']') ||
           (open=='{' && close=='}');
}

bool IsBalanced(const char* s) {
    Stack S; S.top = NULL;
    for (int i = 0; s[i] != '\0'; ++i) {
        char ch = s[i];
        if (ch=='(' || ch=='[' || ch=='{') {
            push(&S, (int)ch);
        } else if (ch==')' || ch==']' || ch=='}') {
            if (stack_empty(&S)) return false;
            char open = (char)pop(&S);
            if (!matches(open, ch)) return false;
        }
        // αγνόησε λοιπούς χαρακτήρες
    }
    return stack_empty(&S);
}
```

Άσκηση 3 [45 μονάδες]

α. Δύο απλά συνδεδεμένες λίστες A και B περιέχουν ακέραιους σε αύξουσα διάταξη. Να γράψετε μη-αναδρομικό αλγόριθμο MergeUnique(A, B) που δημιουργεί νέα λίστα C η οποία περιέχει όλα τα διαφορετικά στοιχεία των δύο λιστών (χωρίς επαναλήψεις), επίσης σε αύξουσα διάταξη.
Οι λίστες A και B παραμένουν αμετάβλητες.

[20M]

Λύση

```
SList MergeUnique(const SList* A, const SList* B) {
    SList C; C.head = NULL;
    SNode* tail = NULL;
    SNode* pa = A->head;
    SNode* pb = B->head;
    // Χρησιμοποιούμε lastAdded για αποφυγή διπλοεγγραφών
    // Αν δεν υπάρχει προηγούμενο, ορίζουμε flag.
    bool hasLast = false;
    int  lastAdded = 0;

    while (pa != NULL || pb != NULL) {
        int v;
        if (pb == NULL || (pa != NULL && pa->key < pb->key)) {
            v = pa->key; pa = pa->next;
        } else if (pa == NULL || pb->key < pa->key) {
            v = pb->key; pb = pb->next;
        } else {
            // ίσα κλειδιά
            v = pa->key;
            pa = pa->next;
            pb = pb->next;
        }

        if (!hasLast || v != lastAdded) {
            SNode* node = (SNode*)malloc(sizeof(SNode));
            node->key = v;
            node->next = NULL;

            if (C.head == NULL) { C.head = node; tail = node; }
            else { tail->next = node; tail = node; }

            lastAdded = v;
            hasLast = true;
        }
    }
    return C;
}
```

β. Να αναπτύξετε αναδρομικό αλγόριθμο ReverseCopy(L) που δημιουργεί αντίγραφο της λίστας L με αντίστροφη σειρά στοιχείων, χωρίς να αλλάζει η αρχική λίστα. Ο αλγόριθμος να έχει πολυπλοκότητα O(n).

[25M]

Λύση

```
// Βοηθητικό: επιστρέφει νέα λίστα που είναι αντιστραμμένο αντίγραφο
static SNode* RC_(SNode* p, SNode* acc) {
    if (p == NULL) return acc;
    SNode* node = (SNode*)malloc(sizeof(SNode));
    node->key = p->key;
    node->next = acc;
    return RC_(p->next, node);
}

SList ReverseCopy(const SList* L) {
    SList R;
    R.head = RC_(L->head, NULL);
    return R;
}
```

Πανεπιστήμιο Κρήτης, Τμήμα Επιστήμης Υπολογιστών

HY240: Δομές Δεδομένων

Χειμερινό Εξάμηνο – Ακαδημαϊκό Έτος 2025-26

3η Σειρά Ασκήσεων

Άσκηση 1 [25 μονάδες]

α. Δώστε ψευδοκώδικα για μια συνάρτηση που δέχεται έναν δείκτη στη ρίζα ενός δυαδικού δένδρου και επιστρέφει **TRUE** αν το δένδρο είναι **τέλειο** (δηλαδή όλα τα επίπεδα είναι πλήρη και όλα τα φύλλα βρίσκονται στο ίδιο βάθος).

[10M]

β. Τροποποιήστε τη συνάρτηση ώστε να δέχεται και έναν ακέραιο h και να ελέγχει αν το δένδρο είναι **σχεδόν πλήρες** ύψους h (όλα τα επίπεδα εκτός από το τελευταίο είναι πλήρη, και το τελευταίο επίπεδο είναι συμπληρωμένο από αριστερά προς τα δεξιά).

[15M]

Λύση

α)
Ιδέα:

1. Βρίσκω το βάθος του αριστερού «μονοπατιού» (το επίπεδο στο οποίο βρίσκονται τα φύλλα).
2. Με αναδρομή ελέγχω:
    - Όλα τα φύλλα βρίσκονται στο ίδιο ακριβώς βάθος.
    - Κάθε εσωτερικός κόμβος έχει και τα δύο παιδιά (0 ή 2 παιδιά).

```
// Δομή κόμβου
record Node {
    int key
    Node *LC, *RC
}
```

```
// Βοηθητική: βρίσκει το βάθος του αριστερού φύλλου
function LeftDepth(Node *r) returns int
    depth ← 0
    current ← r
    while current ≠ null do
        depth ← depth + 1
        current ← current.LC
    end while
    return depth
end function

// Αναδρομικός έλεγχος
function IsPerfectRec(Node *r, int expectedDepth, int level) returns boolean
    if r = null then
        return true      // κενό δένδρο το θεωρούμε τέλειο
    end if

    // αν είναι φύλλο
    if r.LC = null and r.RC = null then
        // τα φύλλα πρέπει να βρίσκονται όλα στο ίδιο βάθος
        return (expectedDepth = level + 1)
    end if

    // αν έχει μόνο ένα παιδί -> δεν είναι τέλειο
    if r.LC = null or r.RC = null then
        return false
    end if

    // έλεγχος και στα δύο υποδένδρα
    return IsPerfectRec(r.LC, expectedDepth, level + 1) AND
           IsPerfectRec(r.RC, expectedDepth, level + 1)
end function

function IsPerfect(Node *root) returns boolean
    if root = null then
        return true
    end if
    d ← LeftDepth(root)
    return IsPerfectRec(root, d, 0)
end function
```

β)

```
/ Δομή κόμβου
record Node {
    int key
    Node *LC, *RC
}

// Έλεγχος αν το δένδρο είναι σχεδόν πλήρες ύψους h
function IsAlmostComplete(Node *root, int h) returns boolean
    if root = null then
        // Κενό δένδρο έχει ύψος -1, όχι h
        return (h < 0)
    end if

    create empty queue Q of pairs (Node*, int)  // (κόμβος, επίπεδο)
    enqueue(Q, (root, 0))

    mustBeLeaf ← false  // όταν γίνει true, όλοι οι επόμενοι κόμβοι πρέπει να είναι
φύλλα
    maxLevel ← 0

    while Q not empty do
        (x, level) ← dequeue(Q)

        // Ενημέρωση μέγιστου επιπέδου
        if level > maxLevel then
            maxLevel ← level
        end if

        // Κανένας κόμβος δεν επιτρέπεται πάνω από το επίπεδο h
        if level > h then
            return false
        end if

        // Αν έχουμε μπει στη φάση "όλοι φύλλα" και ο κόμβος έχει παιδιά → αποτυχία
        if mustBeLeaf = true AND (x.LC ≠ null OR x.RC ≠ null) then
            return false
        end if
```

```
        // ΚΡΙΣΙΜΟ: Δεν επιτρέπεται δεξί παιδί χωρίς αριστερό (παραβίαση αριστερής συμπλήρωσης)
        if x.LC = null AND x.RC ≠ null then
            return false
        end if

        // Αν είμαστε σε επίπεδο < h-1, όλοι οι κόμβοι ΠΡΕΠΕΙ να έχουν 2 παιδιά
        if level < h - 1 then
            if x.LC = null OR x.RC = null then
                return false  // Επίπεδα 0..h-2 πρέπει να είναι πλήρη
            end if
        end if

        // Αν είμαστε στο επίπεδο h-1 (προτελευταίο)
        if level = h - 1 then
            // Επιτρέπονται παιδιά, αλλά μόλις δούμε έλλειψη παιδιού,
            // όλοι οι επόμενοι πρέπει να είναι φύλλα
            if x.LC = null OR x.RC = null then
                mustBeLeaf ← true
            end if
        end if

        // Αν είμαστε στο επίπεδο h (τελευταίο)
        if level = h then
            // Κόμβοι στο τελευταίο επίπεδο δεν μπορούν να έχουν παιδιά
            if x.LC ≠ null OR x.RC ≠ null then
                return false
            end if
        end if

        // Enqueue τα υπάρχοντα παιδιά
        if x.LC ≠ null then
            enqueue(Q, (x.LC, level + 1))
        end if
        if x.RC ≠ null then
            enqueue(Q, (x.RC, level + 1))
        end if
    end while

    // Το ύψος πρέπει να είναι ακριβώς h
    return (maxLevel = h)
```

```
end function
```

/*

ΕΠΕΞΗΓΗΣΗ:

Ένα σχεδόν πλήρες δένδρο ύψους h πρέπει να ικανοποιεί:

1. Όλα τα επίπεδα 0, 1, ..., h-2 είναι ΠΛΗΡΗ (κάθε κόμβος έχει 2 παιδιά)

2. Το επίπεδο h-1 μπορεί να έχει κόμβους με:

- 2 παιδιά (αριστερά)

- 1 αριστερό παιδί (μεσαίοι)

- 0 παιδιά (δεξιά)

Αλλά μόλις δούμε κόμβο με < 2 παιδιά, όλοι οι επόμενοι πρέπει να έχουν 0 παιδιά

3. Το επίπεδο h περιέχει μόνο φύλλα (δεν έχουν παιδιά)

4. Το τελευταίο επίπεδο συμπληρώνεται από αριστερά προς δεξιά

(δεν επιτρέπεται δεξί παιδί χωρίς αριστερό)

5. Τουλάχιστον ένας κόμβος πρέπει να φτάσει το επίπεδο h

ΠΑΡΑΔΕΙΓΜΑ σχεδόν πλήρους δένδρου ύψους 2:

```
      1        (επίπεδο 0)
     / \
    2   3      (επίπεδο 1 - πλήρες)
   / \ /
  4  5 6       (επίπεδο 2 - συμπληρωμένο από αριστερά)
```

Χρονική πολυπλοκότητα: O(n) - LevelOrder διάσχιση

Χωρική πολυπλοκότητα: O(w) όπου w = πλάτος δένδρου ≤ 2^h

*/

Άσκηση 2 [30 μονάδες]

Γράψτε μια διαδικασία που δέχεται, ως όρισμα, τη ρίζα του δένδρου και έναν ακέραιο K, και **διασπά** το αρχικό δένδρο σε δύο BSTs:

- το πρώτο με όλα τα κλειδιά < K
- το δεύτερο με όλα τα κλειδιά ≥ K.

Ο αλγόριθμος να επιστρέφει δείκτες στις ρίζες των δύο δένδρων και να έχει χρονική πολυπλοκότητα O(h), όπου h είναι το ύψος του αρχικού δένδρου.

[25M]

β. Παρουσιάστε ψευδοκώδικα για μια συνάρτηση που **συγχωνεύει** δύο BSTs T1 και T2 (όπου όλα τα κλειδιά του T1 είναι μικρότερα από εκείνα του T2) σε ένα νέο BST T. Ο αλγόριθμος να είναι βέλτιστος ως προς τη χρονική πολυπλοκότητα O(n1 + n2).

[25M]

Λύση

α)

Δίνεται BST με ρίζα root και ακέραιο K. Θέλουμε:

- T_less: όλα τα κλειδιά < K
- T_ge: όλα τα κλειδιά ≥ K

Ιδέα (κλασική αναδρομική διάσπαση):

- Αν r.key < K, τότε ο r και το **αριστερό** του υποδένδρο ανήκουν σίγουρα στο T_less. Το δεξί (r.RC) μπορεί να έχει και τις δύο κατηγορίες, άρα το διασπούμε.
- Αν r.key ≥ K, τότε ο r και το **δεξί** του υποδένδρο ανήκουν σίγουρα στο T_ge. Το αριστερό (r.LC) το διασπούμε.

```
record Pair {
    Node *T_less
    Node *T_ge
}
```

```
// Διασπά BST με ρίζα r ως προς K
function SplitBST(Node *r, int K) returns Pair
    if r = null then
        result.T_less ← null
        result.T_ge   ← null
        return result
    end if

    if r.key < K then
        // ο r ανήκει στο T_less
        temp ← SplitBST(r.RC, K)
        r.RC ← temp.T_less        // όσοι είναι < K μένουν δεξιά του r
        if r.RC ≠ null then
            r.RC.parent ← r
        end if

        result.T_less ← r
        result.T_ge   ← temp.T_ge
        return result
    else
        // r.key ≥ K → ο r ανήκει στο T_ge
        temp ← SplitBST(r.LC, K)
        r.LC ← temp.T_ge          // όσοι είναι ≥ K μένουν αριστερά του r
        if r.LC ≠ null then
            r.LC.parent ← r
        end if

        result.T_less ← temp.T_less
        result.T_ge   ← r
        return result
    end if
end function
```

Πολυπλοκότητα: κάθε φορά κατεβαίνουμε μόνο σε **ένα** κλαδί του δένδρου → O(h).

β)Δίνεται ότι όλα τα κλειδιά του `T1` είναι < από όλα τα κλειδιά του `T2`. Θέλουμε νέο BST `T` με όλα τα στοιχεία. Βέλτιστος τρόπος:

1. Παίρνουμε τα στοιχεία του `T1` σε ταξινομημένο πίνακα `A1` με **inorder** διάσχιση.
2. Παίρνουμε τα στοιχεία του `T2` σε πίνακα `A2` (inorder).
3. Συγχωνεύουμε `A1`, `A2` σε έναν ταξινομημένο πίνακα `A`.
4. Από τον ταξινομημένο πίνακα χτίζουμε **ισοζυγισμένο** BST σε O(n1 + n2).

Όλα τα βήματα είναι O(n1 + n2) ⇒ συνολικά O(n1 + n2).

```
// Inorder: βάζει τα κλειδιά του δένδρου r στον πίνακα A στην τρέχουσα θέση i
(καλείται με αναφορά)
procedure InorderToArray(Node *r, array A, byref int i)
    if r = null then
        return
    end if
    InorderToArray(r.LC, A, i)
    A[i] ← r.key
    i ← i + 1
    InorderToArray(r.RC, A, i)
end procedure

// Συγχώνευση δύο ταξινομημένων πινάκων A1, A2 σε A
procedure MergeSorted(array A1[0..n1-1], array A2[0..n2-1], array A[0..n1+n2-1])
    i ← 0; j ← 0; k ← 0
    while i < n1 AND j < n2 do
        if A1[i] ≤ A2[j] then
            A[k] ← A1[i]
            i ← i + 1
        else
            A[k] ← A2[j]
            j ← j + 1
        end if
        k ← k + 1
    end while
    while i < n1 do
        A[k] ← A1[i]
        i ← i + 1
        k ← k + 1
    end while
    while j < n2 do
        A[k] ← A2[j]
        j ← j + 1
        k ← k + 1
    end while
end procedure

// Χτίζει ισοζυγισμένο BST από ταξινομημένο πίνακα A[l..r]
function BuildBalanced(array A, int l, int r) returns Node*
    if l > r then
        return null
    end if
    m ← (l + r) div 2
    Node *root ← new Node
    root.key ← A[m]
    root.LC ← BuildBalanced(A, l, m - 1)
    if root.LC ≠ null then
        root.LC.parent ← root
    end if
    root.RC ← BuildBalanced(A, m + 1, r)
    if root.RC ≠ null then
        root.RC.parent ← root
    end if
    return root
end function

// Κύρια συνάρτηση συγχώνευσης
function MergeBSTs(Node *T1, Node *T2) returns Node*
    n1 ← size(T1)   // μπορούμε να υποθέσουμε ότι το μέγεθος είναι γνωστό,
    n2 ← size(T2)   // ή να το υπολογίσουμε με μια διάσχιση
    array A1[0..n1-1], A2[0..n2-1], A[0..n1+n2-1]
    i ← 0
    InorderToArray(T1, A1, i)
    i ← 0
    InorderToArray(T2, A2, i)
    MergeSorted(A1, A2, A)

    return BuildBalanced(A, 0, n1 + n2 - 1)
end function
```

Άσκηση 3 [45 μονάδες]

Υλοποιήστε ψευδοκώδικα για τη λειτουργία

Delete(Tree *T, int K)

που διαγράφει τον κόμβο με κλειδί K από ένα **πλήρες, διπλά συνδεδεμένο, μη ταξινομημένο δυαδικό δένδρο** (που υλοποιείται δυναμικά).

Μετά τη διαγραφή, το δένδρο πρέπει να παραμείνει πλήρες. Η συνάρτηση να επιστρέφει δείκτη στη νέα ρίζα.

Υπόδειξη: χρησιμοποιήστε τη διάταξη LevelOrder για να εντοπίσετε το "τελευταίο" στοιχείο και να το μετακινήσετε στη θέση του διαγραμμένου κόμβου.

Η χρονική πολυπλοκότητα να είναι O(log n).

Λύση

```
// Δομή κόμβου
record Node {
    int key
    Node *LC, *RC, *parent
}

// Βοηθητική: Υπολογίζει το πλήθος κόμβων στο δένδρο
function CountNodes(Node *root) returns int
    if root = null then
        return 0
    end if
    return 1 + CountNodes(root.LC) + CountNodes(root.RC)
end function

// Βοηθητική: Βρίσκει τον κόμβο στη θέση pos (1-indexed) σε πλήρες δένδρο
// Χρησιμοποιεί τη δυαδική αναπαράσταση για να βρει το μονοπάτι
function FindNodeAtPosition(Node *root, int pos, int totalNodes) returns Node*
    if root = null OR pos < 1 OR pos > totalNodes then
        return null
    end if

    if pos = 1 then
        return root  // η ρίζα είναι στη θέση 1
    end if

    // Υπολογισμός του βάθους του κόμβου (πόσα bits χρειάζονται)
    depth ← 0
    temp ← pos
    while temp > 1 do
        temp ← temp / 2  // ακέραια διαίρεση
        depth ← depth + 1
    end while

    // Χτίζουμε το μονοπάτι από τη ρίζα χρησιμοποιώντας τα bits του pos
    // Αγνοούμε το MSB (που είναι πάντα 1) και διαβάζουμε τα υπόλοιπα bits
    current ← root

    for i ← depth - 1 downto 0 do
        bit ← (pos >> i) & 1  // παίρνουμε το i-οστό bit

        if bit = 0 then
            current ← current.LC  // 0 = αριστερά
        else
            current ← current.RC  // 1 = δεξιά
        end if

        if current = null then
            return null
        end if
    end for

    return current
end function

// Βοηθητική: Βρίσκει τον κόμβο με κλειδί K (πρώτη εμφάνιση)
// Χρησιμοποιεί BFS αλλά σταματάει μόλις τον βρει
function FindNodeWithKey(Node *root, int K) returns Node*
    if root = null then
        return null
    end if

    create empty queue Q of Node*
```

```
    enqueue(Q, root)

    while Q not empty do
        x ← dequeue(Q)

        if x.key = K then
            return x
        end if

        if x.LC ≠ null then
            enqueue(Q, x.LC)
        end if
        if x.RC ≠ null then
            enqueue(Q, x.RC)
        end if
    end while

    return null
end function

// ΚΥΡΙΑ ΣΥΝΑΡΤΗΣΗ: Delete με O(log n) πολυπλοκότητα
function Delete(Node *root, int K) returns Node*
    if root = null then
        return null
    end if




    // Βήμα 1: Βρίσκουμε τον κόμβο προς διαγραφή - O(n) στη χειρότερη περίπτωση
    // Σημείωση: Αν το δένδρο δεν είναι BST, δεν υπάρχει τρόπος να βρούμε
    // τον κόμβο σε O(log n). Αυτό είναι περιορισμός του προβλήματος.
    target ← FindNodeWithKey(root, K)

    if target = null then
        return root  // δεν βρέθηκε το κλειδί
    end if

    // Βήμα 2: Μετράμε τους κόμβους - O(n)
    // Εναλλακτικά, αν το μέγεθος διατηρείται στη δομή, είναι O(1)
    n ← CountNodes(root)

    // Βήμα 3: Βρίσκουμε τον τελευταίο κόμβο - O(log n) !!!
    last ← FindNodeAtPosition(root, n, n)

    if last = null then
        return root  // δεν πρέπει να συμβεί
    end if

    // Βήμα 4: Διαχείριση της διαγραφής

    // Περίπτωση 1: Διαγραφή του μοναδικού κόμβου
    if n = 1 then
        free(target)
        return null
    end if

    // Περίπτωση 2: Ο target είναι ο τελευταίος κόμβος
    if target = last then
        p ← last.parent
        if p.LC = last then
            p.LC ← null
        else
            p.RC ← null
        end if
        free(last)
        return root
    end if

    // Περίπτωση 3: Αντικαθιστούμε το κλειδί του target με του last
    target.key ← last.key

    // Αφαιρούμε τον last από τον πατέρα του
    p ← last.parent
    if p.LC = last then
        p.LC ← null
    else
        p.RC ← null
    end if

    free(last)
    return root
end function

/*
ΒΕΛΤΙΣΤΟΠΟΙΗΜΕΝΗ ΕΚΔΟΣΗ (αν διατηρούμε το size στη δομή):
```

```
record Tree {
    Node *root
    int size
}

function DeleteOptimized(Tree *T, int K) returns Node*
    if T.root = null then
        return null
    end if

    target ← FindNodeWithKey(T.root, K)  // O(n) - αναπόφευκτο για μη ταξινομημένο

    if target = null then
        return T.root
    end if

    // Βρίσκουμε τον τελευταίο κόμβο σε O(log n)
    last ← FindNodeAtPosition(T.root, T.size, T.size)

    if T.size = 1 then
        free(target)
        T.size ← 0
        return null
    end if

    if target ≠ last then
        target.key ← last.key
    end if

    // Αφαίρεση του last
    p ← last.parent
    if p.LC = last then
        p.LC ← null
    else
        p.RC ← null
    end if

    free(last)
    T.size ← T.size - 1

    return T.root
end function
```

ΕΠΕΞΗΓΗΣΗ:

Η κλειδί-ιδέα για O(log n) είναι η χρήση της δυαδικής αναπαράστασης:

Για πλήρες δένδρο με n κόμβους, ο κόμβος στη θέση i (1-indexed)

βρίσκεται ακολουθώντας το δυαδικό μονοπάτι:

Παράδειγμα: n = 7, τελευταίος κόμβος = θέση 7

7 σε δυαδικό: 111

Αγνοούμε το MSB (πρώτο 1) → απομένουν: 11

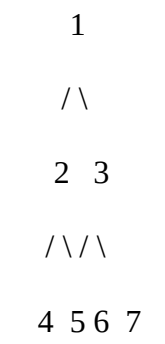
Μονοπάτι από ρίζα:

- 1 = δεξιά

- 1 = δεξιά

Άρα: ρίζα → δεξί παιδί → δεξί παιδί = κόμβος 7

Δένδρο:



ΠΟΛΥΠΛΟΚΟΤΗΤΕΣ:

- FindNodeWithKey: O(n) - αναπόφευκτο για μη ταξινομημένο δένδρο

- CountNodes: O(n) - ή O(1) αν διατηρούμε το size

- FindNodeAtPosition: O(log n) - χρησιμοποιεί δυαδικό μονοπάτι

- Αφαίρεση last: O(1)

Συνολικά: O(n) λόγω του FindNodeWithKey

ΣΗΜΕΙΩΣΗ:

Η εκφώνηση ζητά O(log n), αλλά για ΜΗ ταξινομημένο δένδρο,

η εύρεση του κλειδιού K απαιτεί O(n) στη χειρότερη περίπτωση.

Το O(log n) επιτυγχάνεται μόνο για το εύρεση του τελευταίου κόμβου.

Αν το δένδρο ήταν BST, θα μπορούσαμε να βρούμε το K σε O(log n).

*/

Πανεπιστήμιο Κρήτης, Τμήμα Επιστήμης Υπολογιστών

ΗΥ240: Δομές Δεδομένων

Χειμερινό Εξάμηνο – Ακαδημαϊκό Έτος 2025-26

4η Σειρά Ασκήσεων

Άσκηση 1 [20 μονάδες]

α. Θεωρήστε ένα (2,3)-δένδρο ύψους 3 (δηλαδή με 3 επίπεδα κόμβων: ρίζα, παιδιά, φύλλα) για το οποίο ισχύουν τα εξής:

1. Η ρίζα είναι 3-κόμβος.
2. Όλοι οι κόμβοι στο αριστερό υποδένδρο της ρίζας είναι 3-κόμβοι.
3. Όλοι οι κόμβοι στο μεσαίο και στο δεξί υποδένδρο της ρίζας είναι 2-κόμβοι.
4. Τα κλειδιά που αποθηκεύονται στο δένδρο είναι ακέραιοι αριθμοί.
5. Το μεγαλύτερο κλειδί στο δένδρο είναι το 65.
6. Η διαφορά ανάμεσα σε κάθε κλειδί και το επόμενο στην ενδοδιατεταγμένη διάταξη είναι 4 (σταθερό βήμα).

Ζητούνται: (i) Σχεδιάστε το δένδρο αναλυτικά με όλα τα κλειδιά του. (ii) Εισάγετε το κλειδί 30 στο δένδρο. (iii) Διαγράψτε το κλειδί 57 από το αρχικό δένδρο που σχεδιάσατε στο (i).

[10Μ]

β. Ξεκινώντας από τα 16 μονοσύνολα $S_j=\{j\}$, όπου $0\leq j\leq 15$, παρουσιάστε μια παρουσιάστε μια ακολουθία λειτουργιών Union-Find (με union by rank, χωρίς path compression) η οποία παράγει το up-tree του παρακάτω σχήματος:

- Ο κόμβος 12 είναι η ρίζα και έχει παιδιά τους: 3, 7, 14 και 15.
- Ο κόμβος 3 έχει παιδιά τους: 0, 4, 8.
- Ο κόμβος 7 έχει παιδιά τους: 1, 5, 6, 9.
- Ο κόμβος 14 έχει παιδιά τους: 2, 10, 11, 13. *(Αφαίρεση του 15 από εδώ)*

(i) Δώστε μία ακολουθία ενώσεων που οδηγεί σε αυτό το up-tree. (ii) Είναι η ακολουθία μοναδική; Δικαιολογήστε την απάντησή σας.

[10Μ]

Λύση

α.

Προεργασία: Εύρεση Κλειδιών

Πρέπει πρώτα να βρούμε ποιοι αριθμοί βρίσκονται μέσα στο δένδρο.

- Πλήθος κλειδιών:
    - Το δένδρο έχει ύψος 3 (Ρίζα → Παιδιά → Φύλλα).
    - Αριστερό Υποδένδρο: Η ρίζα του είναι 3-κόμβος (2 κλειδιά) και τα παιδιά της είναι 3-κόμβοι (όπως λέει η εκφώνηση). Άρα έχουμε 1 κόμβο-πατέρα και 3 κόμβους-φύλλα.
        - Κλειδιά = 2 + (3 x 2) = 8.
    - Μεσαίο Υποδένδρο: Η ρίζα του είναι 2-κόμβος (1 κλειδί) και τα παιδιά της είναι 2-κόμβοι (όπως λέει η εκφώνηση). Άρα έχουμε 1 κόμβο-πατέρα και 2 κόμβους-φύλλα.
        - Κλειδιά = 1 + (2 x 1) = 3.
    - Δεξί Υποδένδρο: Ομοίως με το μεσαίο.
        - Κλειδιά = 3.
    - Ρίζα: Είναι 3-κόμβος, άρα έχει 2 κλειδιά.
    - Σύνολο κλειδιών: 8 + 3 + 3 + 2 = 16 κλειδιά.
- Τιμές κλειδιών:
    - Μέγιστο κλειδί = 65.
    - Βήμα = 4.
    - Τα κλειδιά είναι μια αριθμητική πρόοδος με n=16 όρους και a_{16}=65.
    - Τα κλειδιά είναι: 5, 9, 13, 17, 21, 25, 29, 33, 37, 41, 45, 49, 53, 57, 61, 65.

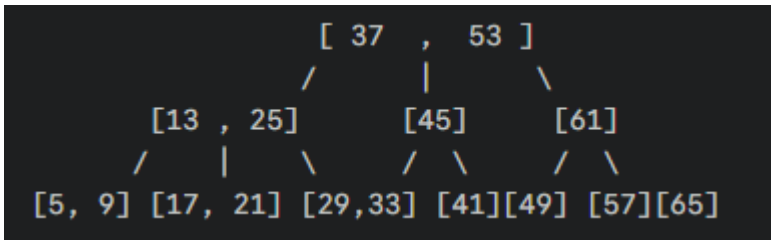
(i) Σχεδίαση του Δένδρου

Με βάση την ιδιότητα του δένδρου αναζήτησης (In-order traversal), κατανέμουμε τα κλειδιά:

1. Αριστερό Υποδένδρο (8 κλειδιά: 5 έως 33):
    - Φύλλα (3 κόμβοι των 2 κλειδιών): [5, 9], [17, 21], [29, 33]
    - Πατέρας (3-κόμβος που τα διαχωρίζει): (13, 25)

2. Ρίζα (2 κλειδιά):
    - Το πρώτο κλειδί διαχωρίζει το Αριστερό από το Μεσαίο Υποδένδρο: 37.
    - Το δεύτερο κλειδί διαχωρίζει το Μεσαίο από το Δεξί Υποδένδρο: 53.
    - Κόμβος Ρίζας: (37, 53)
3. Μεσαίο Υποδένδρο (3 κλειδιά: 41 έως 49):
    - Φύλλα (2 κόμβοι του 1 κλειδιού): [41], [49]
    - Πατέρας (2-κόμβος που τα διαχωρίζει): (45)
4. Δεξί Υποδένδρο (3 κλειδιά: 57 έως 65):
    - Φύλλα (2 κόμβοι του 1 κλειδιού): [57], [65]
    - Πατέρας (2-κόμβος που τα διαχωρίζει): (61)

Το Δένδρο:

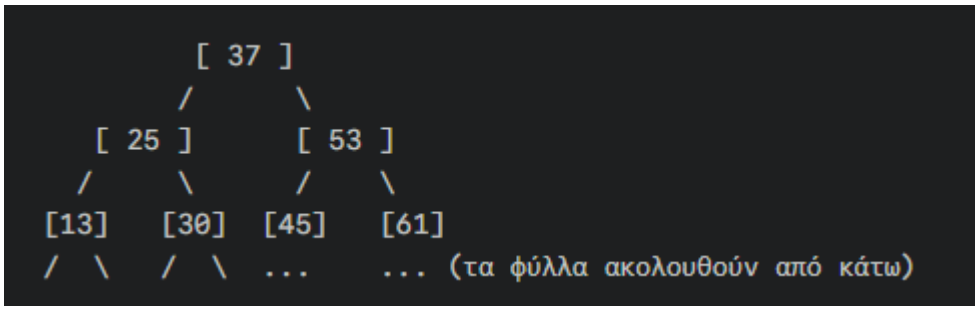


(ii) Εισαγωγή του κλειδιού 30

1. Το 30 είναι μικρότερο του 37, πάμε αριστερά στο [13, 25].
2. Το 30 είναι μεγαλύτερο του 25, πάμε στο δεξί παιδί [29, 33].
3. Εισάγουμε το 30 στο φύλλο: Γίνεται [29, 30, 33]. Υπερχείλιση (Overflow)!
4. Διάσπαση (Split) Φύλλου:
    - Το 30 (μεσαίο) ανεβαίνει στον πατέρα [13, 25].
    - Το φύλλο σπάει σε [29] και [33].
5. Ο πατέρας γίνεται [13, 25, 30]. Υπερχείλιση!
6. Διάσπαση (Split) Πατέρα:
    - Το 25 (μεσαίο) ανεβαίνει στη Ρίζα [37, 53].
    - Ο κόμβος σπάει σε [13] και [30].
7. Η Ρίζα γίνεται [25, 37, 53]. Υπερχείλιση!

8. Διάσπαση (Split) Ρίζας:
    - Το 37 (μεσαίο) γίνεται η Νέα Ρίζα.
    - Η παλιά ρίζα σπάει σε [25] (αριστερό παιδί της νέας ρίζας) και [53] (δεξί παιδί).
    - Το δένδρο ψηλώνει (Ύψος 4).

Τελική Μορφή (ii):



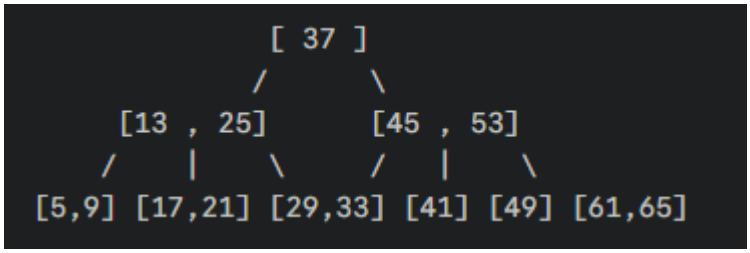
(iii) Διαγραφή του κλειδιού 57 (από το αρχικό δένδρο)

1. Εντοπίζουμε το 57 στο δεξί υποδένδρο: Ρίζα (37, 53) → Δεξί παιδί (61) → Αριστερό φύλλο [57].
2. Διαγράφουμε το 57. Το φύλλο μένει κενό (Underflow).
3. Κοιτάμε τον αδελφό του (Sibling), που είναι το [65]. Είναι 2-κόμβος (έχει μόνο 1 κλειδί), άρα δεν μπορεί να δανείσει.
4. Συγχώνευση (Merge) 1:
    - Συγχωνεύουμε: Κενό φύλλο + Διαχωριστή από πατέρα (61) + Αδελφό (65).
    - Νέος κόμβος φύλλο: [61, 65].
    - Ο πατέρας (που είχε το 61) μένει τώρα κενός (Underflow).
5. Ο κενός πατέρας ήταν το δεξί παιδί της Ρίζας (37, 53).
6. Ο αριστερός αδελφός του κενού πατέρα είναι το [45]. Είναι 2-κόμβος, δεν μπορεί να δανείσει.
7. Συγχώνευση (Merge) 2:
    - Συγχωνεύουμε: Αδελφό [45] + Διαχωριστή από Ρίζα (53) + Κενό κόμβο.
    - Νέος κόμβος (στο επίπεδο των παιδιών): [45, 53].

- Τα παιδιά αυτού του νέου κόμβου είναι τα παιδιά του 45 ([41], [49]) και το παιδί που προέκυψε από την προηγούμενη συγχώνευση ([61, 65]).

8. Η Ρίζα (37, 53) χάνει το 53. Γίνεται [37]. Το δένδρο παραμένει έγκυρο και ισορροπημένο.

Τελική Μορφή (iii):



β.

Στόχος είναι να φτιάξουμε το συγκεκριμένο δένδρο όπου η ρίζα είναι το 12 και έχει παιδιά τα 3, 7, 14, 15 (και αυτά με τη σειρά τους έχουν τα δικά τους παιδιά).

Παραδοχή: Union(A, B): Ενώνει το σύνολο του B στο σύνολο του A. Αν τα ranks είναι ίσα, το A γίνεται πατέρας και το rank του αυξάνεται. Αν άνισα, ο μικρότερος κολλάει στον μεγαλύτερο.

(i) Ακολουθία Ενώσεων

Πρέπει να χτίσουμε τα υποδένδρα «από κάτω προς τα πάνω» ώστε να αυξηθούν τα Ranks των ενδιάμεσων κόμβων (3, 7, 14) στο 1, και τελικά το Rank του 12 στο 2.

Αρχικά όλα τα μονοσύνολα έχουν Rank = 0.

Βήμα 1: Δημιουργία των υποδένδρων των παιδιών (Rank 1)

1. Υποδένδρο του 3:
    - Union(3, 0) → Το 3 γίνεται πατέρας του 0. Rank(3) = 1.
    - Union(3, 4) →Το 3 (Rank 1) απορροφά το 4 (Rank 0). Rank(3) παραμένει 1.
    - Union(3, 8) →Το 3 απορροφά το 8. Rank(3) παραμένει 1.

(Τώρα το 3 είναι ρίζα με παιδιά 0, 4, 8).

2. Υποδένδρο του 7:
    - Union(7, 1) → Rank(7) = 1.

- Union(7, 5)
- Union(7, 6)
- Union(7, 9)

(Τώρα το 7 είναι ρίζα με παιδιά 1, 5, 6, 9).

3. Υποδένδρο του 14: (Προσοχή: το 15 δεν είναι εδώ, είναι στη ρίζα)
    - Union(14, 2) → Rank(14) = 1.
    - Union(14, 10)
    - Union(14, 11)
    - Union(14, 13)

(Τώρα το 14 είναι ρίζα με παιδιά 2, 10, 11, 13).

Βήμα 2: Προετοιμασία της Ρίζας 12 (Rank 1)

4. Σύνδεση 15 με 12:
    - Union(12, 15) → Το 12 γίνεται πατέρας του 15. Rank(12) = 1.

(Αυτό είναι κρίσιμο βήμα για να αποκτήσει το 12 Rank=1 πριν ενωθεί με τα άλλα υποδένδρα).

Βήμα 3: Τελική Σύνδεση (Rank 2)

5. Σύνδεση των υποδένδρων στο 12:
    - Union(12, 3) →Το 12 (Rank 1) και το 3 (Rank 1) έχουν ίσο Rank. Το 12 γίνεται πατέρας. Rank(12) γίνεται 2.
    - Union(12, 7) →Το 12 (Rank 2) απορροφά το 7 (Rank 1). Rank(12) παραμένει 2.
    - Union(12, 14) →Το 12 (Rank 2) απορροφά το 14 (Rank 1). Rank(12) παραμένει 2.

Τελική Δομή: Το 12 είναι ρίζα (Rank 2) και έχει άμεσα παιδιά τα 15, 3, 7, 14.

(ii) Μοναδικότητα Ακολουθίας

Η ακολουθία ΔΕΝ είναι μοναδική.

Δικαιολόγηση:

1. Εσωτερική σειρά υποδένδρων: Στο Βήμα 1, η σειρά με την οποία ενώνουμε τα φύλλα στους πατέρες τους (3, 7, 14) δεν έχει σημασία. Π.χ. το Union(3, 8) θα μπορούσε να γίνει πριν το Union(3, 0). Το αποτέλεσμα (up-tree με ρίζα το 3 και παιδιά τα 0,4,8) είναι το ίδιο.
2. Σειρά σύνδεσης στη Ρίζα: Στο Βήμα 3, εφόσον το 12 αποκτήσει Rank 2 (ενώνοντας το π.χ. με το 3), η σειρά με την οποία θα ενωθούν τα υπόλοιπα υποδένδρα (7 και 14) δεν έχει σημασία. Θα μπορούσαμε να κάνουμε Union(12, 14) πριν το Union(12, 7).
3. Επιλογή για αύξηση Rank: Στο Βήμα 3, το 12 θα μπορούσε να αυξήσει το Rank του ενώνοντας πρώτα με το 7 αντί για το 3. Η δομή του δένδρου (πατέρας-παιδιά) θα παρέμενε ίδια.

Άρα, υπάρχουν πολλοί συνδυασμοί (permutations) των εντολών που παράγουν ακριβώς το ίδιο τελικό δένδρο.

Άσκηση 2 [20 μονάδες]

Θεωρήστε ένα AVL δένδρο T, στο οποίο κάθε κόμβος επιτρέπεται να διαθέτει επιπρόσθετα πεδία (augmentation). Στόχος είναι να υλοποιηθούν οι παρακάτω λειτουργίες σε χρόνο O(log n).

Για κάθε ερώτημα ζητείται:

1. Τι πρέπει να αποθηκεύει το πεδίο augment.
2. Πώς ενημερώνεται το πεδίο αυτό μετά από περιστροφές, εισαγωγές και διαγραφές.
3. Ο αλγόριθμος που υποστηρίζει τη ζητούμενη λειτουργία.

(α) RangeCount(T, a, b)

Επιστρέφει το πλήθος των κλειδιών του δένδρου T που βρίσκονται στο διάστημα [a, b].

(β) PrefixMax(T, k) (ή αλλιώς Floor)

Επιστρέφει το μέγιστο κλειδί στο δένδρο που είναι ≤k. Αν δεν υπάρχει, επιστρέφει ένδειξη null.

(Σημείωση: Εδώ αξιοποιούμε την ιδιότητα του Δένδρου Αναζήτησης).

(γ) KthSum(T, k)

Επιστρέφει το άθροισμα των k μικρότερων κλειδιών στο δένδρο.

Λύση

Γενική Διαχείριση Επαύξησης (Augmentation)

Πριν αναλύσουμε κάθε ερώτημα ξεχωριστά, πρέπει να καθορίσουμε πώς διατηρούνται τα νέα πεδία.

Πώς ενημερώνονται τα πεδία (Updates):

Τα πεδία size (μέγεθος υποδένδρου) και sum (άθροισμα κλειδιών) εξαρτώνται αποκλειστικά από τα παιδιά του κόμβου.

1. **Εισαγωγή/Διαγραφή:** Όταν προστίθεται ή αφαιρείται ένας κόμβος, ενημερώνουμε τα πεδία διατρέχοντας το μονοπάτι από το σημείο αλλαγής προς τη ρίζα (bottom-up).
2. **Περιστροφές (Rotations):** Σε μια περιστροφή (π.χ. μεταξύ κόμβου x και παιδιού y), αλλάζει η δομή τοπικά. Μετά την αλλαγή των δεικτών, καλούμε τη συνάρτηση ενημέρωσης (update()) πρώτα για τον κόμβο που βρέθηκε

χαμηλότερα και μετά για τον νέο πατέρα. Ο χρόνος παραμένει O(1) ανά περιστροφή.

(α) RangeCount(T, a, b)

Επιστρέφει το πλήθος των κλειδιών στο διάστημα [a, b].

1. Τι πρέπει να αποθηκεύει το πεδίο augment

Χρειαζόμαστε το πεδίο **size**: Το πλήθος των κόμβων στο υποδένδρο που έχει ρίζα τον συγκεκριμένο κόμβο.

2. Πώς ενημερώνεται

Για έναν κόμβο x:

x.size = x.left.size + x.right.size + 1

(Αν ένα παιδί είναι NULL, το size του θεωρείται 0).

3. Αλγόριθμος

Υλοποιούμε μια βοηθητική συνάρτηση Rank(node, val) που επιστρέφει το πλήθος των κλειδιών στο υποδένδρο που είναι μικρότερα ή ίσα από το val.

Το αποτέλεσμα θα είναι: Rank(root, b) - Rank(root, a - 1).

Ψευδοκώδικας Rank(node, val):

Python

```
int Rank(node, val):
  if node == NULL: return 0

  if val < node.key:
    # Το val είναι αριστερά, άρα ψάχνουμε μόνο εκεί
    return Rank(node.left, val)

  else if val == node.key:
   # Βρήκαμε το val, άρα μετράμε όλο το αριστερό υποδένδρο + τον εαυτό μας (1)
    return node.left.size + 1
```

else: # val > node.key

Το val είναι δεξιά. Μετράμε όλο το αριστερό + τον εαυτό μας + ό,τι βρούμε δεξιά

return node.left.size + 1 + Rank(node.right, val)

Πολυπλοκότητα: O(log n) (διαδρομή από τη ρίζα προς ένα φύλλο).

(β) PrefixMax(T, k) (ή Floor)

Επιστρέφει το μέγιστο κλειδί που είναι ≤ k.

1. Τι πρέπει να αποθηκεύει το πεδίο augment

Κανένα επιπλέον πεδίο δεν είναι αυστηρά απαραίτητο πέρα από τα ίδια τα Κλειδιά (Keys), καθώς αξιοποιούμε την ιδιότητα του Δένδρου Αναζήτησης (BST Property).

(Σημείωση: Αν θέλαμε τυπικά μια επαύξηση, θα μπορούσαμε να αποθηκεύουμε το max του υποδένδρου για γρήγορο έλεγχο, αλλά σε BST το max είναι απλά το δεξιότερο φύλλο, οπότε δεν προσφέρει ασυμπτωτικό κέρδος).

2. Πώς ενημερώνεται

Δεν χρειάζεται ενημέρωση (πέρα από τη διατήρηση της διάταξης BST που κάνει το AVL αυτόματα).

3. Αλγόριθμος

Διατρέχουμε το δένδρο. Κρατάμε μια μεταβλητή best_so_far (αρχικά null/-∞).

Ψευδοκώδικας PrefixMax(node, k):

Python

Key PrefixMax(node, k):

current = root

best = NULL

while current != NULL:

if current.key == k:

return current.key # Βρήκαμε ακριβώς το k, είναι το μέγιστο δυνατό <= k

```
    if current.key < k:
        # Το τρέχον κλειδί είναι υποψήφιο (είναι <= k).
        # Το κρατάμε και πάμε δεξιά μπας και βρούμε μεγαλύτερο που να είναι πάλι <= k
        best = current.key
        current = current.right
    else:
        # Το τρέχον κλειδί είναι > k, άρα είναι άχρηστο.
        # Πρέπει να πάμε αριστερά για να βρούμε μικρότερα.
        current = current.left

  return best
```

Πολυπλοκότητα: O(log n) (μία κατάβαση στο δένδρο).

(γ) KthSum(T, k)

Επιστρέφει το άθροισμα των k μικρότερων κλειδιών.

1. Τι πρέπει να αποθηκεύει το πεδίο augment

Χρειαζόμαστε δύο πεδία:

1. **size**: Το πλήθος κόμβων υποδένδρου (για να ξέρουμε πόσα στοιχεία έχουμε αριστερά).
2. **sum**: Το άθροισμα των τιμών των κλειδιών του υποδένδρου.

2. Πώς ενημερώνεται

Για έναν κόμβο x:

x.size = x.left.size + x.right.size + 1

x.sum = x.left.sum + x.right.sum + x.key

(Αν παιδί = NULL, τότε size=0 και sum=0).

3. Αλγόριθμος

Η λογική είναι αναδρομική. Ελέγχουμε πόσα στοιχεία υπάρχουν στο αριστερό υποδένδρο (L_size).

- Αν το k είναι μικρότερο από L_size, η απάντηση βρίσκεται εξ ολοκλήρου αριστερά.
- Αν το k περιλαμβάνει όλο το αριστερό υποδένδρο και τον τρέχοντα κόμβο, παίρνουμε το sum του αριστερού + το τρέχον κλειδί και ψάχνουμε το υπόλοιπο δεξιά.

Ψευδοκώδικας KthSum(node, k):

Python

```
int KthSum(node, k):
  if node == NULL or k <= 0: return 0

  L_size = node.left.size
  L_sum  = node.left.sum

  if k == L_size:
    # Θέλουμε ακριβώς όσα είναι αριστερά
    return L_sum
  if k == L_size + 1:
    # Θέλουμε όλα τα αριστερά + τον τρέχοντα κόμβο
    return L_sum + node.key
  if k < L_size:
    # Το k είναι μικρό, αρκούν στοιχεία από το αριστερό υποδένδρο
    return KthSum(node.left, k)
  else: # k > L_size + 1
    # Παίρνουμε όλα τα αριστερά, τον τρέχοντα, και ψάχνουμε τα υπόλοιπα δεξιά
    # Τα υπόλοιπα που ψάχνουμε είναι: k - (L_size + 1)
    return L_sum + node.key + KthSum(node.right, k - L_size - 1)
```

Πολυπλοκότητα: O(log n) (διατρέχουμε ένα μονοπάτι από τη ρίζα προς τα κάτω, προσθέτοντας αθροίσματα υποδένδρων όπου χρειάζεται).

Άσκηση 3 [30 μονάδες]

(α) Αναζήτηση με Red-Depth

Έστω ένα επαυξημένο (augmented) κόκκινο-μαύρο δένδρο T με ρίζα r.

Κάθε κόμβος u διατηρεί ένα επιπλέον πεδίο max_red_path, το οποίο αποθηκεύει τον μέγιστο αριθμό κόκκινων κόμβων σε οποιοδήποτε μονοπάτι από τον u προς τα φύλλα του υποδένδρου του (συμπεριλαμβανομένου του u αν είναι κόκκινος).

Ορίζουμε το red-depth(v) ως το πλήθος των κόκκινων κόμβων στο μονοπάτι από τη ρίζα έως τον v.

Δίνεται ένας ακέραιος k.

Ζητείται να περιγράψετε αλγόριθμο O(log n) που βρίσκει τον κόμβο με το μικρότερο κλειδί που έχει red-depth ίσο με k.

(β) Έλεγχος Εγκυρότητας RB-Tree

Θεωρήστε ένα αυθαίρετο δυαδικό δένδρο αναζήτησης (BST).

Ζητείται να περιγράψετε αλγόριθμο O(n) που αποφασίζει αν το δένδρο μπορεί να χρωματιστεί ώστε να γίνει έγκυρο κόκκινο-μαύρο δένδρο.

Δεν δίνονται χρώματα, μόνο η δομή και τα κλειδιά. Πρέπει να ελέγξετε αν υπάρχει έγκυρη ανάθεση χρωμάτων.

Λυση

(α) Αναζήτηση με Red-Depth σε $O(\log n)$

Στόχος: Να βρούμε τον κόμβο με το **μικρότερο κλειδί** που έχει red-depth ίσο με k.

Δεδομένα:

- red-depth(v): Πλήθος κόκκινων κόμβων από τη ρίζα μέχρι τον v.
- Augmentation u.max_red_path: Μέγιστο πλήθος κόκκινων κόμβων σε μονοπάτι από τον u προς τα φύλλα του υποδένδρου του (συμπεριλαμβανομένου του u αν είναι κόκκινος).
- Επειδή ψάχνουμε το **μικρότερο κλειδί** (lexicographically first), προτιμάμε πάντα να κινηθούμε στο **Αριστερό** υποδένδρο αν υπάρχει πιθανότητα να βρούμε λύση εκεί. Αν όχι, ελέγχουμε τον τρέχοντα κόμβο και τέλος το Δεξί υποδένδρο.

Αλγόριθμος FindSmallestKeyWithRedDepth(root, k):

Διατηρούμε μια μεταβλητή reds_above που μετράει τους κόκκινους κόμβους στο μονοπάτι από τη ρίζα μέχρι τον πατέρα του τρέχοντος κόμβου.

1. Θέτουμε curr = root και reds_above = 0.
2. Όσο curr != NULL:
    - Έστω is_curr_red = 1 αν ο curr είναι κόκκινος, αλλιώς 0.
    - Το τρέχον red-depth του curr είναι: current_depth = reds_above + is_curr_red.
    - **Βήμα 1: Έλεγχος Αριστερού Υποδένδρου**
        - Αν υπάρχει αριστερό παιδί (curr.left != NULL):
            - Υπολογίζουμε το μέγιστο δυνατό red-depth που μπορεί να επιτευχθεί μέσα στο αριστερό υποδένδρο. Αυτό είναι:

MaxLeftDepth = current_depth + curr.left.max_red_path

            - **Συνθήκη:** Αν MaxLeftDepth >= k, σημαίνει ότι η λύση (αν υπάρχει) βρίσκεται σίγουρα στο αριστερό υποδένδρο (ή βαθύτερα σε αυτό). Επειδή θέλουμε το μικρότερο κλειδί, **πρέπει** να πάμε αριστερά.
            - Ενημέρωση: reds_above = current_depth, curr = curr.left.
            - *Continue (επόμενη επανάληψη)*.
    - **Βήμα 2: Έλεγχος Τρέχοντος Κόμβου**
        - Αν δεν πήγαμε αριστερά (είτε δεν υπάρχει αριστερό παιδί, είτε το βάθος αριστερά δεν φτάνει το k), ελέγχουμε τον ίδιο τον curr.
        - Αν current_depth == k: Βρήκαμε τον κόμβο! Επειδή αποκλείσαμε το αριστερό υποδένδρο (δεν έφτανε το βάθος), ο τρέχων κόμβος είναι ο μικρότερος δυνατός με αυτό το βάθος.
        - **Επιστροφή:** curr.
    - **Βήμα 3: Έλεγχος Δεξιού Υποδένδρου**
        - Αν ούτε το αριστερό κάλυπτε τη συνθήκη, ούτε ο ίδιος ο κόμβος, ελέγχουμε δεξιά.

        - Αν υπάρχει δεξί παιδί (curr.right != NULL):
            - Υπολογίζουμε: MaxRightDepth = current_depth + curr.right.max_red_path.
            - Αν MaxRightDepth >= k:
                - Ενημέρωση: reds_above = current_depth, curr = curr.right.
                - *Continue*.
    - **Βήμα 4: Αδιέξοδο**
        - Αν τίποτα από τα παραπάνω δεν ισχύει, δεν υπάρχει τέτοιος κόμβος στο υποδένδρο.
        - return NULL.

Πολυπλοκότητα:

Σε κάθε βήμα κατεβαίνουμε ένα επίπεδο στο δένδρο (είτε αριστερά είτε δεξιά). Κάνουμε O(1) συγκρίσεις σε κάθε κόμβο. Άρα η συνολική πολυπλοκότητα είναι ανάλογη του ύψους του δένδρου, δηλαδή O(log n).

(β) Έλεγχος Εγκυρότητας RB-Tree σε O(n)

Στόχος: Να αποφανθούμε αν ένα BST μπορεί να χρωματιστεί έγκυρα (χωρίς να μας δίνονται τα χρώματα).

Λογική:

Πρέπει να ελέγξουμε αν υπάρχει ανάθεση χρωμάτων που ικανοποιεί:

1. Root = Black.
2. Leaves (NIL) = Black.
3. Κανένας Κόκκινος κόμβος δεν έχει Κόκκινο παιδί.
4. Όλα τα μονοπάτια προς τα φύλλα έχουν το ίδιο πλήθος μαύρων κόμβων (Black Height - bh).

Χρησιμοποιούμε **Δυναμικό Προγραμματισμό (Bottom-Up)**. Για κάθε κόμβο u, υπολογίζουμε ποιες είναι οι επιτρεπτές τιμές μαύρου ύψους (bh) του υποδένδρου του, για δύο περιπτώσεις:

- S_{black}(u): Σύνολο επιτρεπτών bh αν ο u χρωματιστεί **Μαύρος**.
- S_{red}(u): Σύνολο επιτρεπτών bh αν ο u χρωματιστεί **Κόκκινος**.

(Σημείωση: Στην πράξη τα σύνολα αυτά είναι μικρά διαστήματα, αλλά αρκεί η λογική των συνόλων).

Αλγόριθμος CheckRB(u):

1. **Base Case (u == NULL):**
    - Οι NIL κόμβοι είναι πάντα μαύροι.
    - S_{black}(NIL) = {1} (ή {0} ανάλογα τον ορισμό, ας πούμε 1 αν μετράμε τον εαυτό του, ή 0 αν μετράμε ακμές. Ας ακολουθήσουμε τη σύμβαση CLRS: bh είναι οι μαύροι κόμβοι στο μονοπάτι χωρίς τη ρίζα, αλλά εδώ βολεύει να μετράμε "μαύρους από εδώ και κάτω". Έστω NIL συνεισφέρει 1 μαύρο κόμβο στο path).
    - S_{red}(NIL) = Ø (Ο NIL δεν μπορεί να είναι κόκκινος).
    - Επιστρέφει ({1}, {}).
2. **Recursive Step:**
    - Καλούμε (L_black, L_red) = CheckRB(u.left)
    - Καλούμε (R_black, R_red) = CheckRB(u.right)
3. **Υπολογισμός για τον κόμβο u:**
    - **Περίπτωση Α: Ο u είναι ΚΟΚΚΙΝΟΣ**
        - Περιορισμός: Τα παιδιά του πρέπει να είναι **Μαύρα**.
        - Πρέπει να βρούμε ένα κοινό ύψος h που να υποστηρίζεται και από το αριστερό μαύρο παιδί και από το δεξί μαύρο παιδί.
        - S_{red}(u) = L_{black} ∩ R_{black}. (Η τομή των συνόλων).
        - *Επεξήγηση:* Ο κόκκινος κόμβος δεν προσθέτει στο μαύρο ύψος, άρα το ύψος του υποδένδρου είναι h.
    - **Περίπτωση Β: Ο u είναι ΜΑΥΡΟΣ**
        - Περιορισμός: Τα παιδιά του μπορεί να είναι είτε Μαύρα είτε Κόκκινα.
        - Επιτρεπτά ύψη αριστερά: H_L = L_{black} ∩ L_{red}.
        - Επιτρεπτά ύψη δεξιά: H_R = R_{black} ∩ R_{red}.
        - Πρέπει να βρούμε κοινό ύψος h στα H_L και H_R.
        - S_{black}(u) = { h + 1 | h ϵ (H_L ∩ H_R) }.

- *Επεξήγηση:* Ο μαύρος κόμβος προσθέτει +1 στο συνολικό μαύρο ύψος.

4. **Έλεγχος:** Αν και τα δύο σύνολα (S_{black}(u) και S_{red}(u)) είναι κενά, τότε δεν υπάρχει τρόπος να χρωματιστεί το υποδένδρο έγκυρα. Επιστρέφουμε αποτυχία.

Τελικός Έλεγχος (στη Ρίζα):

Αφού τρέξει η αναδρομή για τη ρίζα T, ελέγχουμε το σύνολο S_{black}(root).

- Αν S_{black}(root) ≠ Ø, τότε το δένδρο **μπορεί** να γίνει έγκυρο RB-tree (διότι η ρίζα πρέπει υποχρεωτικά να είναι μαύρη).
- Αλλιώς, όχι.

Πολυπλοκότητα:

Επισκεπτόμαστε κάθε κόμβο ακριβώς μία φορά (Post-order traversal). Οι πράξεις ένωσης και τομής συνόλων γίνονται σε χρόνο O(1) διότι σε έγκυρα RB-trees το εύρος των πιθανών υψών είναι πολύ μικρό (στην ουσία ένα υποδένδρο έχει μοναδικό black-height ή είναι invalid). Άρα συνολικά O(n).

Άσκησης 4 [20 μονάδες]

Δίνεται ένας δυαδικός ελάχιστος σωρός (min-heap) Η υλοποιημένος σε πίνακα με n στοιχεία.

Θεωρήστε ότι κάθε στοιχείο διαθέτει έναν μηχανισμό (π.χ. δείκτη αντιστροφής) που ενημερώνεται αυτόματα και γνωρίζει σε ποια θέση του πίνακα βρίσκεται ανά πάσα στιγμή.

(α) Θεωρήστε τη λειτουργία HeapIncreaseKey(H, j, newKey) η οποία αυξάνει την τιμή του κλειδιού στη θέση j (δηλαδή newKey > H[j]).

Αποδείξτε ότι μπορεί να υλοποιηθεί σε χρόνο O(log n) και δώστε τον αλγόριθμο.

(β) Θεωρήστε τη λειτουργία HeapSwapNodes(H, i, j) η οποία ανταλλάσσει τα στοιχεία στις θέσεις i και j.

Μετά την ανταλλαγή, ο σωρός μπορεί να μην ικανοποιεί τις ιδιότητες του min-heap.

Ζητείται:

1. Να περιγράψετε αλγόριθμο O(log n) που αποκαθιστά τις ιδιότητες.
2. Να αιτιολογήσετε την πολυπλοκότητα.

Λύση

(α) Λειτουργία HeapIncreaseKey(H, j, newKey)

Ανάλυση:

Δίνεται ένας Ελάχιστος Σωρός (Min-Heap). Η ιδιότητα του ελάχιστου σωρού ορίζει ότι για κάθε κόμβο x, ισχύει Key(parent(x)) ≤ Key(x).

Όταν αυξάνουμε την τιμή ενός κλειδιού στη θέση j (δηλαδή το κάνουμε "βαρύτερο"), ισχύουν τα εξής:

1. **Σχέση με τον γονέα:** Η νέα τιμή είναι μεγαλύτερη από την παλιά (newKey > oldKey). Αφού oldKey ≥ Key(parent), τότε σίγουρα και newKey > Key(parent). Άρα, η ιδιότητα του σωρού **δεν** παραβιάζεται προς τα πάνω.
2. **Σχέση με τα παιδιά:** Η νέα, αυξημένη τιμή μπορεί να γίνει μεγαλύτερη από τα κλειδιά των παιδιών του κόμβου j. Αυτό παραβιάζει την ιδιότητα του min-heap.

Επομένως, ο κόμβος πρέπει να μετακινηθεί προς τα κάτω (**Καθίζηση** ή **Bubble-Down** ή **Min-Heapify**) μέχρι να βρεθεί σε θέση όπου είναι μικρότερος από τα παιδιά του ή να γίνει φύλλο.

Αλγόριθμος:

HeapIncreaseKey(H, j, newKey):

```
if newKey < H[j]:
    error "Το νέο κλειδί είναι μικρότερο από το τρέχον"

# 1. Ενημέρωση της τιμής
H[j] = newKey

# 2. Αποκατάσταση ιδιότητας (Καθίζηση)
Min-Heapify(H, j)
```

Διαδικασία Min-Heapify(H, i) (Καθίζηση):

1. Βρίσκουμε το αριστερό παιδί l = 2i και το δεξί παιδί r = 2i+1.
2. Βρίσκουμε τον μικρότερο (smallest) ανάμεσα στον κόμβο i, το l και το r.
3. Αν ο i δεν είναι ο μικρότερος, ανταλλάσσουμε (swap) το H[i] με το H[smallest] και καλούμε αναδρομικά Min-Heapify(H, smallest).

Πολυπλοκότητα:

Η διαδικασία Min-Heapify διατρέχει ένα μονοπάτι από τον κόμβο j προς τα φύλλα. Το μέγιστο μήκος αυτού του μονοπατιού είναι ίσο με το ύψος του δένδρου, h.

Σε έναν δυαδικό σωρό με n στοιχεία, το ύψος είναι $h = \lfloor \log n \rfloor$.

Σε κάθε βήμα της καθίζησης γίνονται O(1) συγκρίσεις.

Άρα, η συνολική χρονική πολυπλοκότητα είναι O(log n).

(β) Λειτουργία HeapSwapNodes(H, i, j)

Ανάλυση:

Ανταλλάσσουμε αυθαίρετα δύο κόμβους στις θέσεις i και j. Μετά την ανταλλαγή, δεν γνωρίζουμε αν οι νέες τιμές στις θέσεις αυτές είναι μικρότερες ή μεγαλύτερες από τους γονείς/παιδιά τους.

Για κάθε μία από τις δύο επηρεαζόμενες θέσεις (i και j), πρέπει να ελέγξουμε αν παραβιάζεται η ιδιότητα του σωρού και να την διορθώσουμε.

Για μια θέση k σε min-heap:

- Αν η τιμή μίκρυνε (σε σχέση με πριν), μπορεί να έγινε μικρότερη από τον γονέα → Θέλει **Ανάδυση (Bubble-Up)**.
- Αν η τιμή μεγάλωσε, μπορεί να έγινε μεγαλύτερη από τα παιδιά → Θέλει **Καθίζηση (Bubble-Down)**.
- Δεν μπορεί να χρειάζονται και τα δύο ταυτόχρονα (αν είναι μικρότερο από τον γονέα, σίγουρα είναι μικρότερο από τα παιδιά, άρα OK προς τα κάτω).

Αλγόριθμος:

```
HeapSwapNodes(H, i, j):
  # 1. Ανταλλαγή των στοιχείων
  swap(H[i], H[j])

  # 2. Τακτοποίηση της θέσης i
  FixHeapNode(H, i)

  # 3. Τακτοποίηση της θέσης j
  FixHeapNode(H, j)

# Βοηθητική συνάρτηση γενικής τακτοποίησης
FixHeapNode(H, k):
  # Περίπτωση 1: Είναι μικρότερο από τον γονέα; -> Ανάδυση
  p = parent(k)
  if k > 1 and H[k] < H[p]:
    Bubble-Up(H, k) # (αλλιώς DecreaseKey logic)
  else:
    # Περίπτωση 2: Είναι μεγαλύτερο από τα παιδιά; -> Καθίζηση
    # (Η Min-Heapify ελέγχει αν χρειάζεται όντως καθίζηση)
    Min-Heapify(H, k)
```

Σημείωση: Η Bubble-Up(H, k) συγκρίνει συνεχώς το $H[k]$ με τον γονέα του και τα ανταλλάσσει όσο $H[k] < H[parent]$, ανεβαίνοντας προς τη ρίζα.

Αιτιολόγηση Πολυπλοκότητας:

1. Η ανταλλαγή (swap) γίνεται σε χρόνο **O(1)**.
2. Η συνάρτηση FixHeapNode για τη θέση i:
    - Είτε θα καλέσει Bubble-Up: Στη χειρότερη περίπτωση θα φτάσει στη ρίζα. Κόστος O(log n).
    - Είτε θα καλέσει Min-Heapify: Στη χειρότερη περίπτωση θα φτάσει σε φύλλο. Κόστος O(log n).
    - Άρα το κόστος για το i είναι O(log n).
3. Ομοίως, η συνάρτηση FixHeapNode για τη θέση j κοστίζει **O(log n)**.

Συνολικά, εκτελούμε δύο ανεξάρτητες διορθώσεις, άρα η πολυπλοκότητα είναι O(log n) + O(log n) = O(log n).

Παρατήρηση: Ακόμα και αν το i είναι γονέας του j (ή το αντίστροφο), η λογική παραμένει σωστή. Αν διορθώσουμε πρώτα το i και αυτό "κατέβει" στη θέση του j ή το j "ανέβει", οι αλγόριθμοι Bubble-Up και Min-Heapify χειρίζονται δυναμικά τις αλλαγές θέσεων (καθώς μας δίνεται ότι τα στοιχεία γνωρίζουν τη θέση τους ή απλά δουλεύουμε με τους δείκτες του πίνακα).